

Lotul Lărgit de Informatică, Seniori, Ziua 2

Comisia științifică

May 21, 2026

Problema 1. Cipizz

Propusă de: Posdărăscu Eugenie-Daniel, Youni, București

În cadrul acestei probleme, vom considera că indexarea se face de la zero.

Răspunsul pentru această problemă este egal cu numărul total de triunghiuri care se pot plasa în matrice, din care se scad numărul de triunghiuri care conțin cel puțin un o celulă blocată pe cel puțin una dintre laturi.

Pentru numărul total de triunghiuri există mai multe moduri de a ajunge la o formulă finală. Unul dintre acestea este să scriem numărul de triunghiuri ca $\sum_{i=1}^n \sum_{j=1}^m \min(i, j)$. Acum, putem viziona matricea rezultată de această expresie și să o împărțim în trei zone, iar pentru fiecare putem exprima suma folosind formula lui Gauss. Într-un final, ajungem la o formulă de genul $\frac{n \cdot (n+1) \cdot (3 \cdot m - n + 1)}{6} - n \cdot m$ (unde presupunem că $n \leq m$).

Partea mai dificilă din problemă este să calculăm numărul de triunghiuri care conțin cel puțin o celulă blocată pe perimetru. În cele ce urmează, vom nota laturile unui triunghi cu h , v , respectiv d , după cum urmează în figura de mai jos.

.	.	.	.	.	.	.	.	.
.	h	.	.	.	.	.	.	.
.	h	d	.	.	.	.	.	.
.	h	.	d	.	.	.	.	.
.	h	.	.	d	.	.	.	.
.	h	.	.	.	d	.	.	.
.	h	.	.	.	.	d	.	.
.	v	v	v	v	v	v	d	.
.	.	.	.	.	.	.	.	.

Vom folosi principiul includerii și al excluderii: pentru fiecare subset de laturi ale triunghiuri, vom calcula numărul de triunghiuri care conțin câte o celulă blocată pe fiecare dintre laturile din acel subset.

Latura h

Pentru fiecare dintre cele k puncte, vrem să numărăm câte triunghiuri există care conțin acest punct pe latura h . Mai mult decât atât, nu vrem să numărăm același triunghi de mai multe ori, așa că vom vrea ca acest punct să fie cel mai de sus punct blocat de pe latura h .

Să notăm punctul fixat cu $P = (x, y)$. Vrem să vedem unde putem pune vârful de sus al triunghiului, dar și ce lungime trebuie să aibă laturile sale. Trebuie, deci, să vedem cât de mult ne putem extinde, atât în sus, cât și în jos pe latura h plecând din P .

- în jos - $n - x - 1$
- în sus - $\min(Q.x - x)$, unde Q este un punct blocat cu $Q.y = y$. Atenție, punctele din afara matricei sunt și ele considerate blocate.

Să notăm cu up cât de mult ne putem extinde în sus și cu $down$ cât de mult ne putem extinde în jos. Lungimea laturii triunghiului va fi $up + down$. Însă, vrem ca toate colțurile triunghiului să fie în interiorul matricei. Așadar, vrem să numărăm soluțiile (a, b) astfel încât $1 \leq a \leq up$, $1 \leq b \leq down$, $a + b \leq m - y$.

Vom începe prin a număra soluțiile (a, b) ale $1 \leq a$, $1 \leq b$, $a + b \leq n$. Răspunsul este $\frac{n(n-1)}{2}$. Să notăm cu $f(n)$ acest rezultat.

Răspunsul pentru problema inițială se poate afla folosind din nou principiul includerii și al excluderii: numărăm soluțiile fără constrângerile pe up și $down$, din care scădem soluțiile care au $a > up$, scădem soluțiile cu $b > down$, iar la final adăugăm soluțiile cu $a > up$ și $b > down$. Prin urmare, rezultatul este $f(m - y) - f(m - y - up) - f(m - y - down) + f(m - y - down - up)$.

Reușim, astfel, să rezolvăm acest caz în $O(k)$ timp.

Latura v

Vom rezolva foarte similar cu cazul anterior. Pentru un punct fixat, vrem să numărăm triunghiuri care conțin acest punct pe latura de jos, iar acesta este cel mai din stânga punct blocat. Astfel, ne putem extinde:

- în stânga - $\min(y - Q.y)$, unde Q este blocat și $Q.x = x$.
- în dreapta - $m - y - 1$

Mai mult, lungimea laturii trebuie să fie mai mică sau egală cu $x + 1$. Avem, deci, nevoie de exact aceeași procedură ca la cazul anterior.

Latura d

Similar, pentru un punct $P = (x, y)$, vrem să numărăm triunghiurile care conțin punctul P pe latura d , iar P este cel mai de jos punct de pe această latură. Așadar, ne putem extinde:

- în stânga-sus - $\min(x, y)$
- în dreapta-jos - $\min(Q.x - x)$, unde Q este blocat și $x - y = Q.x - Q.y$

De vreme ce fixăm diagonala, avem garanția că celelalte colțuri ale triunghiului există, deci, nu avem restricții pe $up + down$.

Laturile h și d

Pentru acest caz, vom fixa două puncte blocate, P_1 și P_2 . P_1 se va afla pe latura h , iar P_2 se va afla pe latura d . Mai mult, P_1 este cel mai de jos punct blocate de pe latura h , iar P_2 este cel mai de jos punct de pe latura d .

Prin urmare, $P_1.y < P_2.y$. Vom calcula punctul de pe h unde s-ar intersecta diagonală punctului P_2 . Acesta este $P_3 = (P_2.x - (P_2.y - P_1.y), P_1.y)$. Dacă P_3 nu se află în matrice sau $P_3.x > P_1.x$, atunci putem ignora această pereche.

Altfel, aceste două puncte ne vor da un interval valid de lungimi ale triunghiurilor cu colțul de sus în P_3 . Prin urmare, trebuie să determinăm lungimea minimă și cea maximă a unei laturi valide:

- lungimea minimă - $\max(P_1.x - P_3.x + 1, P_2.x - P_3.x)$
- lungimea maximă - $\min(P_1.x + \text{dist_to_next_on_}h_{P_2} - P_3.x, P_2.x + \text{dist_to_next_on_}d_{P_2} - P_3.x - 1)$

Răspunsul pentru această pereche de puncte, este, deci, $\max(0, \text{lungimea maximă} - \text{lungimea minimă} + 1)$.

Rezolvăm, deci, acest caz în $O(k^2)$.

Laturile v și d

Pentru acest caz, vom fixa două puncte blocate, P_1 și P_2 . P_1 se va afla pe latura v , iar P_2 se va afla pe latura d . Mai mult, P_1 este cel mai din stânga punct blocate de pe latura v , iar P_2 este cel mai de sus punct blocate de pe latura d .

Prin urmare, $P_1.x \geq P_2.x$. Vom calcula punctul de pe v unde s-ar intersecta diagonală punctului P_2 . Acesta este $P_3 = (P_1.x, P_2.y + P_1.x - P_2.x)$. Dacă P_3 nu se află în matrice sau $P_3.y \leq P_1.y$, atunci putem ignora această pereche.

Altfel, aceste două puncte ne vor da un interval valid de lungimi ale triunghiurilor cu colțul din dreapta-jos în P_3 . Prin urmare, trebuie să determinăm lungimea minimă și cea maximă a unei laturi valide:

- lungimea minimă - $\max(P_3.y - P_1.y, P_3.y - P_2.y + 1)$
- lungimea maximă - $\min(P_3.y - P_1.y + \text{dist_to_prev_on_}v_{P_1} - 1, P_3.y - P_2.y + \text{dist_to_prev_on_}d_{P_2})$

Răspunsul pentru această pereche de puncte este, deci, $\max(0, \text{lungimea maximă} - \text{lungimea minimă} + 1)$.

Rezolvăm, deci, acest caz în $O(k^2)$.

Laturile v și h

Pentru acest caz, vom fixa două puncte blocate, P_1 și P_2 . P_1 se va afla pe latura h , iar P_2 se va afla pe latura v . Mai mult, P_1 este cel mai de sus punct blocate de pe latura h , iar P_2 este cel mai din stânga punct blocate de pe latura v .

Prin urmare, $P_1.x < P_2.x$ și $P_1.y \leq P_2.y$. Vom calcula punctul P_3 , colțul din stânga-jos al triunghiului (unde h se întâlnește cu v). Acesta este $P_3 = (P_2.x, P_1.y)$. Deoarece ambele coordonate provin de la puncte valide din matrice, P_3 se află întotdeauna în interiorul matricei.

Aceste două puncte ne dau un interval valid de lungimi ale triunghiurilor cu colțul din stânga-jos în P_3 :

- lungimea minimă - $\max(P_3.x - P_1.x, P_2.y - P_3.y + 1)$
- lungimea maximă - $\min(P_3.x - P_1.x + \text{dist_to_prev_on_}h_{P_1} - 1, P_2.y - P_3.y + \text{dist_to_next_on_}v_{P_2})$

Răspunsul pentru această pereche de puncte este, deci, $\max(0, \text{lungimea maximă} - \text{lungimea minimă} + 1)$.

Rezolvăm, deci, acest caz în $O(k^2)$.

Toate cele trei laturi

Vom începe prin a fixa punctul D , care se va afla pe latura d . Pentru fiecare D fixat, vrem să numărăm perechile de puncte (H, V) astfel încât H se află pe latura h (și este cel mai de sus punct blocat de pe acea latură), iar V se află pe latura v .

Colțurile triunghiului rezultat sunt complet determinate de cele trei puncte. Colțul de sus este $A = (D.x - (D.y - H.y), H.y)$ (intersecția coloanei lui H cu diagonală lui D), iar colțul din dreapta-jos este $C = (V.x, D.y + V.x - D.x)$ (intersecția rândului lui V cu diagonală lui D).

```

. . . . .
.  A . . . . .
.  h  d  . . . . .
.  H .  d  . . . . .
.  h  . .  d  . . . . .
.  h  . . .  D . . . . .
.  h  . . . .  d  . . . . .
.  B v  V v  v  v  C .
. . . . .

```

Observația cheie este că $H.y < D.y$ (deoarece H se află pe latura h , la stânga diagonalei). Dacă sortăm toate punctele crescător după coordonata y , atunci în momentul în care procesăm un punct ca potențial V , toate punctele care ar putea fi H (cu y mai mic) au fost deja văzute.

Algoritmul este, deci, următorul: pentru fiecare D fixat, parcurgem celelalte puncte j în ordinea sortată. La fiecare pas, încercăm punctul j în ambele roluri:

parcursere după y												
$\rightarrow$												
.	.	.	.	.	.	.	.	.	.	.	.	.
.	A	.	.	.	.	.	.	.	.	.	.	.
.	h	d	.	.	.	.	.	.	.	.	.	.
.	H	.	d	.	.	.	.	.	.	.	.	.
.	h	.	.	d	.	.	.	.	.	.	.	.
.	h	.	.	.	D	.	.	.	.	.	.	.
.	h	.	.	.	.	d	.	.	.	.	.	.
.	B	v	V	v	v	v	C	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.	.	.	.

$H.y = 1 < D.y = 5$

$\Rightarrow$ adăugat în BIT

$V.y = 3, V.x \geq D.x$

$\Rightarrow$ interoghează BIT

1. **Ca H (punct pe h).** Condițiile sunt:

- $D.y > H.y$
- $A = (D.x - (D.y - H.y), H.y)$ se află în matrice și $A.x \leq H.x$
- H este cel mai de sus punct blocat de pe h : $H.x - \text{dist_to_prev_on_h}_H + 1 \leq A.x$

Dacă toate sunt satisfăcute, adăugăm H într-un arbore Fenwick, indexat după coordonata x normalizată.

2. **Ca V (punct pe v).** Condițiile sunt:

- $V.x \geq D.x$ și $V \neq D$
- $C = (V.x, D.y + V.x - D.x)$ se află în matrice și $C.y \geq V.y$
- V poate ajunge la C pe v : $V.y + \text{dist_to_next_on_v}_V \geq C.y$
- D poate ajunge la C pe d : $D.y + \text{dist_to_next_on_d}_D - 1 \geq C.y$

Dacă toate sunt satisfăcute, interogăm arborele Fenwick pentru numărul de puncte H cu $H.x < V.x$ (adică H se află deasupra lui V , ceea ce este necesar geometric) și adăugăm rezultatul la răspuns.

Condiția $H.x < V.x$ este verificată eficient prin interogarea arborelui Fenwick pe prefixul coordonatelor x normalizate.

Complexitatea totală este $O(k^2 \log k)$: k iterații pentru D , iar pentru fiecare D , o parcursere de k puncte cu operații de arbore Fenwick în $O(\log k)$.

Problema 2. Hipermetropie

Propusă de: stud. Prosie Radu-Teodor, Universitatea din București

Subtask 1 ($M \leq 2$)

Pentru acest subtask, distingem 2 cazuri distincte, în funcție de numărul de muchii: dacă avem o singură muchie, atunci ambii supuși din nodurile incidente muchiei vor răspunde cu **Da** în prima rundă. Dacă avem două muchii, atunci dacă ele formează un lanț, supusul din mijlocul lanțului va răspunde cu **Da** în prima rundă, iar dacă cele două muchii nu au niciun nod comun, atunci supușii care au o muchie incidentă vor răspunde toți cu **Da** în runda 2 (exact al doilea caz din exemple). Complexitatea este $O(N)$.

Subtask 2 (gradul unui nod este maxim 1)

Pentru acest subtask, vedem că vom avea M perechi de supuși care se comportă identic. Inductiv, observăm că jocul se termină în M runde, iar toți supușii ce au o muchie adiacentă cu nodul lor răspund cu **Da**. Complexitatea este $O(N)$.

Subtask 3 ($N \leq 20$)

Observăm că singurul tip de informație pe care îl primește un supus este că jocul nu s-a terminat încă. Atunci, el va acționa astfel: deduce care este numărul de runde în care s-ar termina jocul dacă nu ar avea nicio muchie adiacentă, fie acest număr X , și va răspunde cu **Da** în runda $X + 1$. Astfel, soluția pentru acest subtask ia o formă recursivă: pentru un anumit graf $G = (V, E)$, căutăm $ans = \min_{x \in V} Z(x)$, unde $Z(x)$ este numărul de runde în care se termină jocul dacă eliminăm toate muchiile lui x . Vor răspunde cu **Da** în runda $ans + 1$ toți supușii care minimizează Z . Cazul de bază este un graf fără muchii, pentru care considerăm că jocul se termină în 0 runde. Complexitatea este $O(N \cdot 2^N)$.

Subtask-urile 4 și 5

Ne uităm mai atent la ce face, de fapt, algoritmul de la subtask-ul 3. Observăm că ceea ce ne interesează este să alegem nodurile ale căror muchii incidente să fie eliminate astfel încât să ajungem cât mai repede la un graf fără muchii, ceea ce reprezintă, de fapt, un *minimum vertex cover*. Numărul de runde va fi $|MVC|$, iar supușii care răspund cu **Da** sunt cei care fac parte din cel puțin un *minimum vertex cover*. Aceste informații pot fi aflate cu o dinamică simplă, calculată separat pentru fiecare nod, în complexitate $O(N^2)$ pentru subtask-ul 4, sau în $O(N)$ cu o dinamică ce folosește *rerooting*, pentru punctaj complet.

Problema 3. Stiva

Propusă de: stud. Bogdan Vlad-Mihai, Universitatea din București

Să rulăm algoritmul funcției `compress` pe șirul s .

Să considerăm starea stivei după primii i pași ai algoritmului, trecând la pasul $i + 1$. Fie c caracterul cu care trebuie să actualizăm stiva. Dacă caracterul c este egal cu vârful stivei, atunci starea stivei devine cea de la pasul $i - 1$. Altfel, vom adăuga caracterul c la stivă, atingând o stare nouă a stivei - starea de la pasul i , la care se aplică litera c .

Pe baza acestei descrieri, putem observa că actualizarea stivei cu caracterul c este similară cu traversarea unei muchii într-un graf, unde fiecare nod corespunde stării stivei pe un anumit prefix al șirului s .

Pentru a fi mai exacti, putem observa că acest graf este un arbore. Mai mult, operația `pop` ne duce în părintele nodului, iar `push c` ne duce într-un copil al nodului curent, traversând o muchie pe care o vom nota cu eticheta c . Prin urmare, putem privi acest arbore ca o trie, unde nu avem direcții pe muchii. Să notăm această trie cu T .

Pentru moment, vom considera o trie (T') peste același alfabet, cu două proprietăți auxiliare:

- fiecare nod din această trie are **exact** m vecini (evident, câte unul pentru fiecare etichetă de la 0 la $m - 1$).
- are un număr infinit de noduri, care se extind în "orice direcție". Cu alte cuvinte, T' este un fractal.

Acum, pornind dintr-un nod oarecare u al lui T' , vrem să vedem unde (și cum) ajungem dacă aplicăm funcția `compress` pe un șir t . Pentru fiecare literă din t , vom merge din nodul curent în vecinul său către care are o muchie etichetată cu litera c . Putem observa că o succesiune de două litere consecutive egale ne va ține, practic, pe loc. Mai mult, putem să extindem această observație și să concluzionăm că orice șir care se compresează la șirul vid ne va ține pe loc.

Prin urmare, singurele operații care ne depărtează de nodul u sunt cele care adaugă litere ce nu vor fi șterse de alte litere din dreapta lor. Fie v nodul în care ajungem după ce am parcurs întregul șir t .

Pe baza observațiilor de mai sus, se poate concluziona ușor că drumul de la u la v va lista exact starea stivei la finalul procedurii `compress(t)`. Prin urmare, rezultatul `compress(t)` este distanța dintre u și v .

Întorcându-ne la tria T , observăm că singura proprietate pe care aceasta trebuie să o aibă pentru ca întregul raționament de mai sus să se aplice identic este ca drumul plecând de la un i (cu $0 \leq i < n$) până la orice j (cu $i \leq j < n$) să aibă toate tranzițiile definite (deci, să nu vrem să mergem pe o muchie care nu există în T). Acest lucru este clar adevărat, de vreme ce pentru a ajunge la nodul care corespunde stării stivei la pasul j , am trecut anterior prin nodul care corespunde stării stivei la pasul i , pentru că șirul a fost parcurs de la stânga la dreapta.

Prin urmare, lungimea `compress(s[i...j])` este egală cu lungimea drumului între nodul care corespunde stării stivei după $i - 1$ pași și nodul care corespunde stivei după j pași.

Rămâne, deci, să calculăm suma distanțelor într-un arbore între toate perechile de noduri (unde unele noduri se pot "suprapune"). Această problemă se poate rezolva ușor calculând contribuția fiecărei muchii.

În funcție de structurile folosite pentru reprezentarea lui T , complexitatea finală poate să fie $O(n)$ sau $O(n \log n)$. Diverse alte abordări duc la algoritmi cu complexitatea timp $O(n)$ sau $O(n \log n)$. O soluție $O(n)$ obține punctajul maxim, însă o soluție $O(n \log n)$ implementată suficient de atent poate obține, de asemenea, punctaj maxim.

Echipa

Problemele pentru acest baraj au fost pregătite de:

- Adrian Panaete, Colegiul Național "A.T. Laurian", Botoșani
- Nicoli Marius, Colegiul Național "Frații Buzești" Craiova
- Frâncu Cătălin, Nerdvana
- Tamio-Vesa Nakajima, Universitatea Marburg, Hesse
- Andrei-Costin Constantinescu, ETH, Zürich, Elveția
- Alexandru Lorintz, Universitatea "Babeș-Bolyai", Cluj-Napoca
- Toma Ariciu, Universitatea Politehnică București
- Andrei-Cristian Ivan, Universitatea Politehnică București
- Dan-Constantin Spătărel, București
- Vlad-Mihai Bogdan, Universitatea București
- Eugenie-Daniel Posdărăscu, Youni, București
- Radu-Teodor Prosie, Universitatea București
- Liviu Armand Gheorghe, Universitatea București
- Tiberiu Cozma, Universitatea Alexandru Ioan Cuza, Iași
- Ion Andrei Robert, Ecole Polytechnique, Paris
- Cismaru Mihaela, Google Paris
- Perju Verzotti Luca, Ecole Polytechnique, Paris
- Gheorghies Alexandru, Universitatea "Alexandru Ioan Cuza" Iași
- Boacă Andrei, Universitatea "Alexandru Ioan Cuza" Iași
- Daniel Gheorghe, Universitatea din București
- Arsenoiu Iulian, Columbia University
- Feodorov Andrei, ETH, Zürich, Elveția
- Botnaru Victor, Universitatea Politehnica Bucuresti, ACS
- Popescu Adrian, Universitatea Politehnica Bucuresti, ACS
- Măgureanu Livia, JetBrains
- Popa Bogdan-Ioan, Just 4 Kids
- Andrei Chertes, Universitatea Politehnică București